

Mean-Variance Portfolio Optimization via Active-Set Quadratic Programming

Tarik Agić

Student ID: 20108

Numerical Optimization Course Project

Abstract—This paper presents a complete implementation of long-only mean-variance portfolio optimization as a convex quadratic program. Periodic asset returns are loaded from a validated comma-separated-value matrix and used to estimate the sample mean-return vector and unbiased sample covariance matrix. For a specified target return, the optimizer minimizes portfolio variance subject to full investment, the target-return equality, and nonnegative asset weights. The numerical core is a primal active-set method. It starts from an explicitly constructed feasible portfolio, solves an equality-constrained Karush–Kuhn–Tucker system at every iteration, truncates a step when a bound becomes active, and removes an active bound when its multiplier violates the optimality conditions. The implementation uses the `natID` framework for the graphical interface and for two interchangeable linear-system backends: `dense::DblMatrix` and `sparse::DblSolver` with LU factorization. Repeated solutions from the global minimum-variance return to the largest feasible asset return form the efficient frontier. The application also visualizes individual assets, the selected portfolio, changes of the active set, and portfolio weights across target returns. On the bundled five-asset, 24-period data set, the global minimum-variance portfolio achieves a periodic return of 0.726733% at 0.134594% risk. At a 1% target, the solver converges in six iterations with two active bounds and 0.281134% risk. Dense and sparse solutions agree to approximately machine precision. Automated tests additionally cover analytical cases, feasibility and KKT conditions, invalid inputs, frontier construction, and two 20-asset systems.

Index Terms—active-set method, efficient frontier, mean-variance optimization, quadratic programming, portfolio optimization, sparse linear systems

I. INTRODUCTION

Mean-variance portfolio selection expresses the allocation of capital as a trade-off between expected return and risk. Markowitz’s formulation established that portfolio risk depends not only on the variances of individual assets but also on their covariances [1]. Consequently, combining assets whose returns do not move identically can yield a portfolio with lower variance than any allocation derived from individual volatilities alone.

This project treats portfolio construction as a numerical optimization problem rather than as a financial forecasting exercise. The inputs are a matrix of periodic asset returns, and the outputs are optimal portfolio weights for one or more prescribed returns. Short selling is excluded, every unit of available capital must be invested, and no transaction costs, taxes, or risk-free asset are modeled. Under these assumptions the problem has a quadratic objective, two linear equalities, and bound constraints. Because a sample covariance matrix

is positive semidefinite, the resulting program is convex; any point satisfying the Karush–Kuhn–Tucker (KKT) conditions is a global solution.

The main numerical challenge is the changing set of zero-weight assets. An asset with $w_i = 0$ has an active lower bound. As the requested return changes, assets enter and leave this active set, producing different allocation regimes along the efficient frontier. Active-set quadratic-programming methods are well suited to this structure because they explicitly identify the binding constraints and solve a sequence of equality-constrained subproblems [2], [3].

The completed application contains four connected stages. First, it validates and loads return observations. Second, it estimates the mean vector and covariance matrix. Third, it solves either a global minimum-variance problem or a target-return problem by an active-set method. Finally, it constructs and visualizes the efficient frontier and the corresponding weight paths. The software is written in C++20 and uses `natID` both for its native graphical interface and for dense and sparse matrix solvers [6]. A common matrix-backend interface permits the same active-set algorithm to be executed with either solver, making numerical agreement directly measurable.

The contribution of the project is therefore not a new financial model, but a transparent end-to-end realization of the model: data validation, statistical estimation, constrained optimization, backend comparison, visualization, export, and automated numerical verification are implemented in one program.

II. PROBLEM FORMULATION

A. Return Statistics

Let T denote the number of observation periods and n the number of assets. The input matrix $\mathbf{R} \in \mathbb{R}^{T \times n}$ stores the periodic return R_{ti} of asset i in period t . For each asset, the estimator of expected periodic return is

$$\mu_i = \frac{1}{T} \sum_{t=1}^T R_{ti}. \quad (1)$$

The implementation uses the unbiased sample covariance estimator

$$\Sigma_{ij} = \frac{1}{T-1} \sum_{t=1}^T (R_{ti} - \mu_i)(R_{tj} - \mu_j). \quad (2)$$

The standard deviation $\sigma_i = \sqrt{\Sigma_{ii}}$ is used when an individual asset is drawn on the risk-return diagram. All quantities remain on the scale of one observation period; the application does not silently annualize input data.

For a portfolio weight vector $\mathbf{w} \in \mathbb{R}^n$, expected return and variance are

$$\mu_p(\mathbf{w}) = \boldsymbol{\mu}^T \mathbf{w}, \quad (3)$$

$$\sigma_p^2(\mathbf{w}) = \mathbf{w}^T \boldsymbol{\Sigma} \mathbf{w}, \quad (4)$$

and the displayed portfolio risk is $\sigma_p(\mathbf{w}) = \sqrt{\sigma_p^2(\mathbf{w})}$.

B. Long-Only Target-Return Program

For a requested periodic return r , the application solves

$$\begin{aligned} & \underset{\mathbf{w}}{\text{minimize}} && \frac{1}{2} \mathbf{w}^T \boldsymbol{\Sigma} \mathbf{w} \\ & \text{subject to} && \boldsymbol{\mu}^T \mathbf{w} = r, \\ & && \mathbf{1}^T \mathbf{w} = 1, \\ & && \mathbf{w} \geq \mathbf{0}. \end{aligned} \quad (5)$$

The factor $1/2$ does not change the minimizer, but makes the objective gradient equal to $\boldsymbol{\Sigma} \mathbf{w}$. The budget equality enforces full investment, while nonnegativity enforces a long-only portfolio. With only these constraints, the feasible target interval is

$$r \in \left[\min_i \mu_i, \max_i \mu_i \right]. \quad (6)$$

Targets outside this interval are rejected before a linear system is formed.

The global minimum-variance (GMV) problem removes the target-return equality from (5). Its solution is the left endpoint of the efficient branch of the risk-return curve. The application first computes this portfolio and then samples target values uniformly between its expected return and $\max_i \mu_i$. Solving (5) at those targets yields the displayed efficient frontier. Sampling begins at the GMV return rather than at $\min_i \mu_i$, because lower-return minimum-variance portfolios belong to the inefficient branch.

III. ACTIVE-SET NUMERICAL METHOD

A. Feasible Initialization

A primal active-set method must begin with a feasible point. For an interior target r , the implementation finds one asset with the largest mean below r and one with the smallest mean above r . If their returns are $\mu_\ell < r < \mu_u$, a two-asset feasible portfolio is obtained from

$$w_\ell = \frac{\mu_u - r}{\mu_u - \mu_\ell}, \quad w_u = 1 - w_\ell, \quad (7)$$

with all remaining weights equal to zero. Equation (7) satisfies both equalities and nonnegativity by construction. At a boundary of the feasible interval, the solver restricts the problem to all assets whose means match that boundary and computes their minimum-variance allocation. The GMV solve starts from equal weights $w_i = 1/n$.

B. Working Set and KKT Direction

Let $\mathcal{W}_k$ be the set of zero-weight bounds treated as active at iteration k , and let $\mathbf{E}_{\mathcal{W}_k}$ select their components. The equality matrix and right-hand side are

$$\mathbf{A} = \begin{bmatrix} \mathbf{1}^T \\ \boldsymbol{\mu}^T \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 1 \\ r \end{bmatrix} \quad (8)$$

for a target-return solve. The GMV version contains only the first row.

At a feasible iterate $\mathbf{w}_k$, the gradient is $\mathbf{g}_k = \mathbf{H} \mathbf{w}_k$, where $\mathbf{H}$ is a regularized covariance matrix described below. The search direction is obtained by solving the equality-constrained quadratic subproblem. Its KKT equations are

$$\begin{bmatrix} \mathbf{H} & \mathbf{A}^T & \mathbf{E}_{\mathcal{W}_k}^T \\ \mathbf{A} & \mathbf{0} & \mathbf{0} \\ \mathbf{E}_{\mathcal{W}_k} & \mathbf{0} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{p}_k \\ \boldsymbol{\lambda}_k \\ \boldsymbol{\nu}_k \end{bmatrix} = \begin{bmatrix} -\mathbf{g}_k \\ \mathbf{0} \\ \mathbf{0} \end{bmatrix}. \quad (9)$$

The last block forces $p_i = 0$ for active assets, while $\mathbf{A} \mathbf{p}_k = \mathbf{0}$ preserves the equalities. Direct factorization of (9) is appropriate for the moderate systems considered here; nevertheless, the system is indefinite and can become ill-conditioned when asset returns or covariance columns are nearly dependent [4].

C. Blocking Bounds and Multiplier Test

If $\mathbf{p}_k$ is nonzero, the full step is accepted unless it would make a free weight negative. The step length is

$$\alpha_k = \min \left(1, \min_{\substack{i \notin \mathcal{W}_k \\ p_{k,i} < 0}} \frac{-w_{k,i}}{p_{k,i}} \right). \quad (10)$$

The update $\mathbf{w}_{k+1} = \mathbf{w}_k + \alpha_k \mathbf{p}_k$ remains feasible. When $\alpha_k < 1$, the blocking weight is set exactly to zero and its bound is added to the working set.

When $\|\mathbf{p}_k\|_\infty$ is below the direction tolerance, the current point minimizes the objective over the present working set. The multipliers of the active bounds are then inspected. An active bound whose multiplier has the wrong sign is removed, permitting that weight to increase in a later iteration. If no multiplier violates the sign condition, the KKT conditions for the full long-only problem are satisfied and the iteration terminates. This add-or-remove logic is what allows the algorithm to follow different active-set regions of the efficient frontier.

D. Regularization and Stopping Checks

The matrix used inside (9) is symmetrized and slightly regularized:

$$\mathbf{H} = \frac{1}{2} (\boldsymbol{\Sigma} + \boldsymbol{\Sigma}^T) + \delta \mathbf{I}, \quad (11)$$

$$\delta = \max(10^{-12}, \rho \max(10^{-12}, s)), \quad (12)$$

$$s = \frac{1}{n} \sum_{i=1}^n |\Sigma_{ii}|, \quad (13)$$

where the default relative regularization is $\rho = 10^{-9}$. Regularization is used only to stabilize the numerical systems. Reported variance and risk are recomputed with the original sample covariance matrix.

The default maximum is 500 active-set iterations. The primal, direction, and multiplier tolerances are respectively 10^{-9} , 10^{-10} , and 10^{-9} . After termination, the implementation independently checks equality residuals and nonnegativity. A solve is reported as converged only when these checks pass. This prevents a successful factorization from being confused with a valid optimization result.

IV. SOFTWARE ARCHITECTURE AND NATID INTEGRATION

The implementation separates data processing, numerical optimization, and presentation. `CsvReader` parses the return matrix and reports the line and column of invalid entries. It supports quoted CSV fields and an optional observation-label column named `Date`, `Period`, `Time`, `Timestamp`, or `Observation`. Asset names must be nonempty and unique, all numeric values must be finite, and the matrix must contain at least two assets and two observations. `StatisticsCalculator` implements (1) and (2). The structures `ReturnData`, `Statistics`, `PortfolioSolution`, and `EfficientFrontier` carry the resulting data between modules.

`ActiveSetQPSolver` implements the method of Section III and does not depend on GUI controls. Linear algebra is delegated to `MatrixBackend`, which exposes one interface for two professor-provided `natID` implementations:

- `dense::DblMatrix` stores the complete KKT matrix and solves it through the dense matrix API;
- `sparse::DblSolver` receives only nonzero triplets and uses general LU factorization with multi-pass Markowitz pivoting and the solver’s own ordering strategy.

Although the mathematical KKT matrix is symmetric, the sparse path deliberately uses the general LU mode. This avoids assuming positive definiteness, which an indefinite saddle-point matrix does not possess. Sparse direct methods can exploit structural zeros in larger systems, but their setup and ordering costs mean that sparsity does not automatically imply lower runtime for a small portfolio [5].

`AppState` owns the loaded data and current results. It resets stale solutions whenever a new data set is loaded, invokes the selected backend, constructs summaries, compares both backends, and exports the frontier. The export contains target and achieved returns, risk, variance, iteration count, active-constraint names, and every asset weight at full double precision.

The `natID` interface contains separate views for return data, optimization, the efficient frontier, and weight paths. The optimization view disables data-dependent controls until a valid matrix has been loaded and adjusts the numeric target limits to the feasible interval in (6). The risk-return chart draws individual assets, the efficient frontier, and a distinct marker for the most recently optimized portfolio. The weight chart plots $w_i(r)$ for every asset and marks sampled positions where the active set changes. English and Bosnian resources use identical translation keys. A CSV format panel documents the expected input directly inside the application.

TABLE I
BUNDLED DATA SET: SAMPLE STATISTICS

Asset	Mean (%)	Std. dev. (%)
Global Equity	0.837500	1.524243
Technology	1.266667	2.446411
Government Bonds	0.516667	0.703202
Corporate Bonds	0.720833	0.358717
Gold	0.995833	1.258529

TABLE II
AUTOMATED VERIFICATION SCENARIOS

Scenario	Verified property
CSV and statistics	Dimensions, known sample means, covariance symmetry, and malformed-number rejection.
Two-asset target	Analytical $w_1 = w_2 = 0.5$ solution at the prescribed return.
Diagonal GMV	Analytical weights (36, 9, 4)/49.
Active bound	Primal feasibility, a zero-weight asset, and reduced-gradient KKT conditions.
Infeasible target	Rejection above the long-only feasible interval.
20-asset diagonal	Dense and sparse feasibility and agreement of all weights.
20-asset correlated	Positive-definite $BB^T + D$ covariance, dense/sparse agreement, and an interior optimum.
Efficient frontier	25 feasible, monotonically ordered solutions on the bundled data set.

V. EXPERIMENTAL SETUP AND VERIFICATION

A. Bundled Demonstration Data

The bundled deterministic data set contains 24 monthly observations for five synthetic asset classes. Synthetic observations are appropriate for a reproducible demonstration: every evaluator obtains identical statistics, frontier points, and active-set transitions. They are not presented as a forecast or as evidence of investable performance. Table I lists the statistics produced by the application; percentages are per observation period.

The sample intentionally contains materially different risk-return profiles and strong positive and negative correlations. This produces a visible diversification effect and causes several bound constraints to change status across the frontier. The example is therefore more informative for the active-set method than a diagonal covariance matrix, while remaining small enough for manual interpretation.

B. Automated Numerical Tests

The executable `PortfolioCoreTests` is registered with `CTest`. Its checks are summarized in Table II. The tests exercise both successful and rejected inputs; they do not merely verify that the application runs without an exception.

The KKT test reconstructs the objective gradient with the original covariance matrix. Multipliers for the budget and return equalities are inferred from two free assets. Reduced gradients must be near zero for positive weights and must have the correct sign at active lower bounds. The larger correlated problem uses $\Sigma = BB^T + D$ with strictly positive diagonal D , which guarantees positive definiteness while retaining nonzero cross-asset correlations.

TABLE III
OPTIMAL PORTFOLIO WEIGHTS

Asset	GMV (%)	Target 1% (%)
Global Equity	11.483536	0.000000
Technology	0.000000	24.316306
Government Bonds	23.552014	0.000000
Corporate Bonds	50.205170	22.432726
Gold	14.759280	53.250968
Total	100.000000	100.000000

A separate backend check solves

$$\begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 5 \\ 7 \end{bmatrix} \quad (14)$$

with both natID implementations and verifies the known result $(x, y) = (1.6, 1.8)$.

VI. RESULTS AND DISCUSSION

A. Efficient Frontier and Allocations

With 25 sampled frontier points, the GMV portfolio has expected periodic return 0.726733%, risk 0.134594%, and variance 1.811551×10^{-6} . The highest feasible sampled target is 1.266667%, equal to the mean of Technology. Table III compares the GMV allocation with the independently optimized 1% target.

The GMV solution assigns its largest weight to Corporate Bonds, the asset with the smallest individual standard deviation, but it still allocates capital to three other assets because covariance determines portfolio risk. Technology is excluded at the GMV point. Raising the target to 1% changes the balance: Technology supplies the higher expected return, Gold becomes the largest holding, and the lower-return Global Equity and Government Bonds reach their active lower bounds. The 1% portfolio attains the target to numerical precision, has risk 0.281134%, variance approximately 7.90×10^{-6} , and converges in six active-set iterations.

Across the 25 sampled points, five active-set transitions create six observed allocation regions. The sequence begins with Technology fixed at zero near the GMV portfolio, passes through an interior region, and then successively fixes Global Equity, Government Bonds, and Corporate Bonds as the target increases. At the maximum target, Technology has weight one and all other assets are active. These changes explain the visible piecewise behavior of the weight paths and changes in curvature of the sampled frontier.

Figure 1(a) is intended to be read with risk on the horizontal axis and expected return on the vertical axis. A point above and to the left is preferable in the mean-variance sense, although the model does not select a single universally best point because investors may accept different risk levels. Figure 1(b) exposes information that the frontier alone cannot show: which assets generate each efficient portfolio and exactly where zero-weight constraints change.

TABLE IV
DENSE-SPARSE COMPARISON AT A 1% TARGET

Quantity	Dense	Sparse
Iterations	6	6
Active constraints	2	2
Single-run time (ms)	0.6823	0.3571
Budget residual	0	0
Return residual	1.735×10^{-18}	1.735×10^{-18}
Max. weight difference: 5.551×10^{-17}		
Variance difference: 5.082×10^{-21}		

B. Dense and Sparse Backend Agreement

Table IV reports one comparison captured in the Debug configuration for the 1% target. Both backends execute the same active-set logic and differ only in the solution of (9).

The active sets are identical, and the maximum weight difference is close to double-precision roundoff. This is stronger evidence than matching objective values alone because two implementations independently reproduce the entire allocation. The comparison tolerance used by the interface is 10^{-7} . The measured times are displayed for diagnostic interest only. They are single runs from a Debug build, include solver setup, and do not control processor state or repeat count; they therefore do not establish a general performance advantage for either storage scheme.

The sparse representation becomes conceptually more relevant as the number of assets and active bounds grows, because the lower KKT blocks contain many structural zeros. However, the covariance block estimated from ordinary asset returns is generally dense. Consequently, a portfolio KKT matrix is not fully sparse, and a performance claim requires controlled experiments over multiple problem sizes. The project deliberately limits its conclusion to numerical agreement and correct use of both professor-provided matrix implementations.

VII. LIMITATIONS

The numerical results must be interpreted within the model assumptions. Historical or synthetic sample means are uncertain estimates of future return, and sample covariance can be poorly conditioned when the observation count is small relative to the number of assets. A long-only restriction avoids leveraged and short positions but does not model upper bounds, sector limits, turnover, transaction costs, taxes, or a risk-free asset. Variance penalizes positive and negative deviations symmetrically and does not directly represent tail loss. The model is single-period and static; it does not rebalance through time.

The bundled data are synthetic and were chosen for deterministic verification and clear visualization. They demonstrate the numerical method, not a trading strategy. A practical deployment would require a documented historical data source, frequency and currency normalization, treatment of missing values, out-of-sample evaluation, and sensitivity analysis. From a numerical perspective, larger or nearly dependent data sets may also require stronger covariance estimation, scaling,

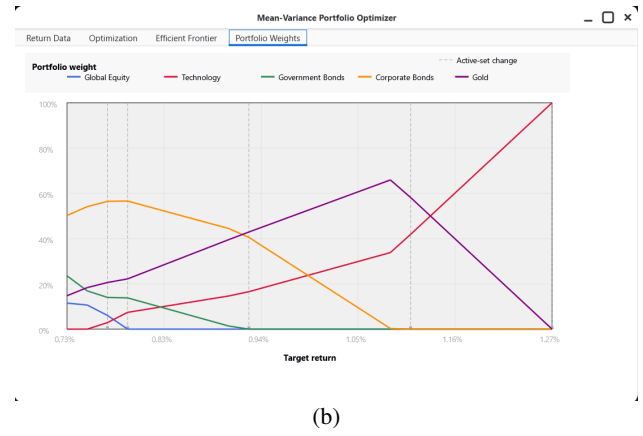
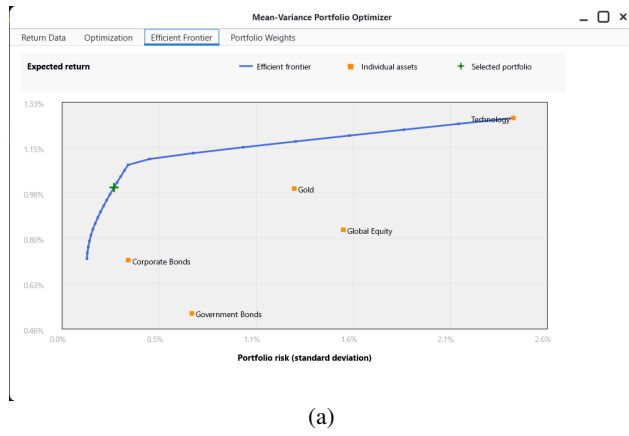


Fig. 1. Portfolio visualizations for the bundled five-asset data set: (a) the efficient frontier, individual assets, and selected 1% portfolio; (b) portfolio weights across target returns, with dashed markers at sampled active-set changes.

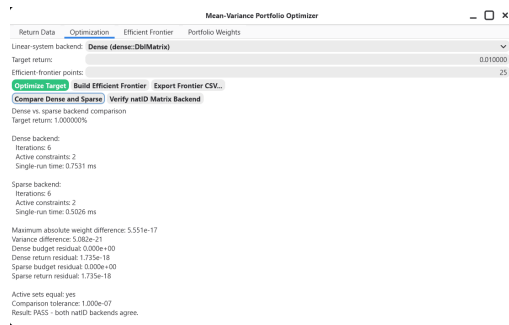


Fig. 2. Optimization view showing the numerical comparison of the two natID matrix backends.

or specialized symmetric-indefinite factorization. These extensions are intentionally outside the scope of the implemented course project.

VIII. CONCLUSION

The project implements the proposed mean-variance model as a complete and verifiable numerical application. It estimates return statistics from a CSV matrix, formulates the long-only target-return problem as a convex quadratic program, and solves it through a feasible primal active-set method. Explicit working-set updates make zero-weight constraints observable rather than hiding them behind a library optimizer.

Repeated solutions construct the efficient frontier and show how allocations change as additional return is required.

The implementation also demonstrates a separation between optimization logic and linear algebra. Dense and sparse natID backends solve the same KKT systems and agree to near machine precision on the bundled example, while automated tests extend verification to analytical and 20-asset cases. The resulting frontier, weight paths, residuals, active constraints, and CSV export provide multiple independent views of correctness. Within its stated long-only, single-period assumptions, the application fulfills the intended numerical optimization objectives and provides a clear basis for explaining both the mathematics and the implementation.

REFERENCES

- [1] H. Markowitz, "Portfolio selection," *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, Mar. 1952, doi: 10.2307/2975974.
- [2] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed. New York, NY, USA: Springer, 2006.
- [3] D. Goldfarb and A. Idnani, "A numerically stable dual method for solving strictly convex quadratic programs," *Mathematical Programming*, vol. 27, pp. 1–33, 1983, doi: 10.1007/BF02591962.
- [4] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed. Baltimore, MD, USA: Johns Hopkins University Press, 2013.
- [5] T. A. Davis, *Direct Methods for Sparse Linear Systems*. Philadelphia, PA, USA: Society for Industrial and Applied Mathematics, 2006.
- [6] I. Džafić, "natID: Native Interface Design for macOS, Linux, and Windows," GitHub repository. [Online]. Available: <https://github.com/idzafic/natID>. Accessed: Sep. 18, 2026.